

Stratax V2 Benchmark Report

Cross-Platform and Compiler Performance Analysis

Noah Ryan

September 24, 2026

Abstract

This report evaluates the performance characteristics of Stratax, a scientific array library implemented in C++. Benchmark Suite V2 measures representative array operations across multiple hardware, operating-system, and compiler configurations.

Each benchmark configuration uses repeated warmup executions, timed samples, and independent trials to measure both runtime and measurement variability.

The purpose of this report is to document the benchmark methodology, present the resulting measurements, identify performance trends, and establish a reproducible baseline for future optimization and regression testing.

1 Objective

The primary objective of Benchmark Suite V2 is to characterize the runtime performance of Stratax under different execution environments.

The experiment considers several questions:

- How does runtime scale as the number of array elements increases?
- How much does compiler choice affect performance on the same processor?
- How does the operating environment affect performance?
- Are performance differences consistent across different operations?
- How reproducible are repeated benchmark measurements?

The results are intended to provide a baseline against which future changes to Stratax can be compared.

2 Methodology

2.1 Benchmark Procedure

Each benchmark case follows the same general procedure.

First, an untimed correctness validation is performed. The benchmark operation is then executed 10 times as a warmup before measurements begin.

Each trial contains 50 timed executions of the operation. A total of 5 independent trials are performed for each platform and compiler configuration.

Timing is performed using:

`std::chrono::steady_clock`

The timed interval contains the benchmarked operation and any result allocation performed by that operation.

Input construction, correctness validation, checksum computation, result destruction, and console output are excluded from the timed interval.

2.2 Problem Sizes

The default benchmark suite uses three square array sizes.

Dimensions	Elements
64×64	4,096
256×256	65,536
1024×1024	1,048,576

Table 1: Array sizes used by Benchmark Suite V2.

2.3 Benchmark Configuration

The default benchmark parameters are summarized in Table 2.

Parameter	Value
Benchmark suite version	V2
Warmup executions	10
Timed samples per trial	50
Independent trials	5
Primary timing statistic	Median
Clock	<code>std::chrono::steady_clock</code>

Table 2: Benchmark methodology parameters.

3 Experimental Platforms

Benchmark Suite V2 was executed on multiple hardware and software configurations.

Processor	OS	Compiler	Configuration
Intel Core Ultra 9 288V	Debian	GCC	Laptop
Intel Core Ultra 9 288V	Debian	Clang	Laptop
Ryzen 9 9950X	Ubuntu	GCC	Desktop
Ryzen 9 9950X	Ubuntu	Clang	Desktop
Ryzen 9 9950X	Windows	Clang	Desktop
Ryzen 9 9950X	Windows	MSVC	Desktop

Table 3: Hardware, operating-system, and compiler configurations.

Exact compiler versions and additional environment metadata are stored alongside the benchmark results in the corresponding JSON files.

4 Benchmark Workloads

Benchmark Suite V2 contains several operations intended to exercise different parts of the Stratax array implementation.

4.1 Iteration

The iteration benchmarks measure traversal over owning arrays and array views.

`owning_iteration` Sequential iteration over an owning array.

`contiguous_view_iteration` Iteration over a contiguous array view.

`strided_view_iteration` Iteration over a non-contiguous or strided array view.

4.2 Arithmetic and Broadcasting

These benchmarks measure elementwise operations and broadcasting.

`owning_scalar_add` Adds a scalar value to each element of an owning array.

`owning_row_broadcast_add` Adds a row vector to an owning matrix using broadcasting.

4.3 Reductions

Reduction benchmarks measure operations that combine many array elements into smaller results.

`owning_axis0_sum` Sum reduction along axis 0.

`owning_axis1_sum` Sum reduction along axis 1.

`strided_view_axis1_sum` Axis-1 reduction over a strided view.

4.4 Materialization and Copying

These benchmarks measure operations that produce new owning storage from existing arrays or views.

`contiguous_view_materialize` Materializes a contiguous view into owning storage.

`strided_view_materialize` Materializes a strided view into owning storage.

`owning_to_tensor_copy` Copies an owning array into a tensor representation.

5 Statistical Analysis

Performance measurements are affected by operating-system scheduling, interrupts, background activity, cache state, and other sources of variability. Multiple statistics are therefore used to describe the measurements.

5.1 Median

The median is used as the primary timing statistic because it is less sensitive to isolated timing spikes than the arithmetic mean.

For an ordered set of measurements

$$x_1, x_2, \dots, x_n,$$

the median represents the central measurement.

5.2 Arithmetic Mean

The arithmetic mean is calculated as

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (1)$$

5.3 Sample Standard Deviation

The sample standard deviation is

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}. \quad (2)$$

This statistic measures the spread of repeated timing measurements around their mean.

5.4 Median Absolute Deviation

Median absolute deviation, or MAD, is calculated as

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|). \quad (3)$$

MAD provides a measure of spread that is less sensitive to outliers than standard deviation.

5.5 Coefficient of Variation

The coefficient of variation is

$$\text{CV} = \frac{s}{\bar{x}} \times 100\%. \quad (4)$$

CV expresses variability relative to the mean runtime, making it easier to compare noise between benchmarks with different absolute runtimes.

5.6 Across-Trial Statistics

Each benchmark configuration is executed for 5 independent trials.

The plotted runtime is the median of the individual trial medians:

$$T = \text{median}(m_1, m_2, \dots, m_5), \quad (5)$$

where m_i is the median runtime from trial i .

Error bars represent the sample standard deviation of the trial medians.

5.7 Relative Performance

When comparing two configurations, relative speedup may be calculated as

$$S = \frac{T_{\text{baseline}}}{T_{\text{comparison}}}. \quad (6)$$

A value of

$$S > 1$$

indicates that the comparison configuration completed the operation in less time than the baseline.

6 Results

This section presents selected benchmark results. The complete set of generated plots is available in the `benchmarks/plots/` directory.

6.1 Iteration Performance

Figure 1 shows an example comparison of contiguous view iteration across the tested configurations.

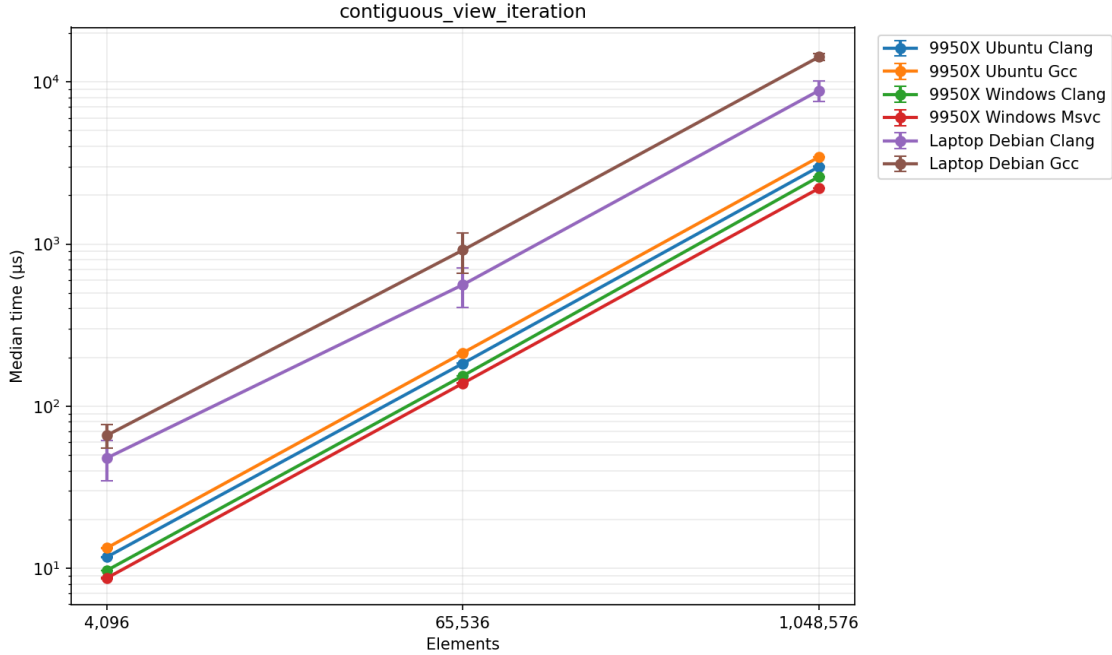


Figure 1: Contiguous view iteration performance. Each point represents the median of the trial medians. Error bars represent the standard deviation of trial medians.

6.2 Arithmetic Performance

6.3 Broadcasting Performance

6.4 Reduction Performance

6.5 Materialization and Copy Performance

6.6 Selected Numerical Results

Table 4 demonstrates one way to present a small subset of exact measurements.

Configuration	Elements	Median (μ s)	Trial CV (%)
Ubuntu GCC	1,048,576	0.0	0.0
Ubuntu Clang	1,048,576	0.0	0.0
Windows Clang	1,048,576	0.0	0.0
Windows MSVC	1,048,576	0.0	0.0

Table 4: Example format for reporting selected benchmark statistics. Replace the placeholder values with measurements from the generated summary data.

7 Discussion

The results should be interpreted by considering both runtime and measurement variability.

This section should focus on patterns rather than simply repeating values from the tables.

Useful questions include:

- Does runtime scale proportionally with the number of elements?
- Do compiler differences remain consistent across problem sizes?
- Do different operations respond differently to compiler choice?
- Are there benchmarks where one configuration behaves unexpectedly?
- Are observed differences much larger than the measured trial-to-trial variation?

Large or unexpected differences should be treated as observations that may require additional investigation rather than immediately being assigned a cause.

8 Limitations

Benchmark Suite V2 provides controlled and repeatable measurements, but several limitations remain.

- The laptop and desktop use different processor architectures, memory systems, thermal limits, and power-management behavior.
- Comparisons between Windows and Linux also involve differences in runtime libraries, standard-library implementations, ABI, and system behavior.

- CPU affinity is not explicitly controlled.
- CPU frequency and thermal state are not directly fixed.
- Background operating-system activity may affect timing.
- Extremely short operations can be difficult to measure precisely when timed individually.
- Benchmark Suite V2 measures Stratax internally and does not compare against external numerical libraries such as NumPy.

9 Future Work

Future benchmark studies may investigate:

- multithreaded execution,
- SIMD/vectorization,
- larger problem sizes,
- cache and memory-bandwidth behavior,
- additional processors,
- release-to-release performance regressions,
- comparisons with NumPy and other numerical libraries.

10 Conclusion

Benchmark Suite V2 establishes a reproducible performance baseline for Stratax across multiple compiler, operating-system, and processor configurations.

The results provide both performance measurements and estimates of measurement variability. These measurements can be used to guide optimization work and to identify future performance regressions.

A Benchmark Parameters

Parameter	Value
Benchmark suite	V2
Warmups	10
Samples per trial	50
Trials	5
Small size	64×64
Medium size	256×256
Large size	1024×1024

Table 5: Benchmark Suite V2 parameters.

B Reproduction

A benchmark trial can be executed using the benchmark driver.

```
python scripts/benchmark.py \  
    --trial 1 \  
    --compiler g++ \  
    --output results.json
```

Statistical analysis can be regenerated using:

```
python benchmarks/analyze.py
```

Plots can be regenerated using:

```
python benchmarks/plot.py
```

C Repository Data

The complete machine-readable benchmark results are stored in:

```
benchmarks/results/v2/
```

Generated figures are stored in:

```
benchmarks/plots/v2/
```

Statistical summaries are stored in:

```
benchmarks/analysis/v2/
```